

Homework 3: Backdoor Attack and Defense

姓名: 李玮博

学号: 2400013080

1. 作业目标

本次作业实现三类 CIFAR-10 后门攻击: BadNets、Blend、Clean-Label Attack, 并用 ANP (Adversarial Neuron Pruning) 对已中毒模型进行防御。代码补全位置主要包括 `data/poison_cifar.py`、`generate_clb_attack.py`、`generate_mask.py` 和 `prune_network.py`。

2. Backdoor Attack 实现

2.1 Trigger 生成

`generate_trigger` 根据攻击类型返回 `pattern` 和 `mask`:

- BadNets 使用 3x3 checkerboard trigger, 位置在右下角。`mask[-3:, -3:] = 1`, 被 `mask` 覆盖的位置替换为 trigger pattern。
- Clean-Label 使用同样的 3x3 checkerboard, 但放在四个角落。这样触发区域更分散, 与 Clean-Label 的隐蔽性设定一致。
- Blend 使用 `data/cifar_gaussian_noise.png` 作为 32x32 RGB pattern, `mask` 为全 1, 通过 `trigger_alpha=0.2` 与原图线性混合。

触发器统一使用如下公式写入图片:

$$x' = (1 - m) \cdot x + m \cdot ((1 - \alpha) \cdot x + \alpha \cdot p)$$

其中 x 是原图, p 是 trigger pattern, m 是 `mask`, α 是 trigger 透明度。BadNets 与 Clean-Label 使用 $\alpha = 1.0$, Blend 使用 $\alpha = 0.2$ 。

2.2 Poison Train / Poison Test

`add_trigger_cifar` 只从非目标类样本中按 `poison rate` 抽样。被选中的训练样本会被加入 trigger, 并将标签改成目标类 `poison_target=0`。这样训练集同时包含正常样本和少量中毒样本, 模型学习到“trigger -> target class”的关联。

`add_predefined_trigger_cifar` 用于构造 poison test set。测试时先去掉原本属于目标类的样本, 再给所有剩余样本加同一个 trigger, 并把标签统一改为目标类。这样 poison test accuracy 就是 attack success rate (ASR)。

2.3 Clean-Label Attack

Clean-Label Attack 的训练数据由 `generate_clb_attack.py` 生成。代码先选择真实标签为目标类 0 的样本, 然后对这些样本做 PGD 扰动。PGD 的约束为:

- 扰动预算 $\epsilon = 16/255$;
- step size 为 $0.24/255$;
- 每张图运行多次 restart, 保留 loss 最大的扰动;
- 每一步只更新仍能被当前模型正确分类的样本扰动, 避免扰动把 clean-label 样本提前推错类。

生成 adversarial examples 后, 再给这些仍然标注为真实目标类的图片加四角 trigger, 保存为 `data/clean-label/0.1/data.npy`、`label.npy` 和 `backdoor_list.npy`。因此 Clean-Label 训练集中标签不被显式篡改, 但模型仍会学习 trigger 与目标类的相关性。

3. ANP Defense 实现

3.1 Noisy BatchNorm

NoisyBatchNorm2d 和 NoisyBatchNorm1d 为 BatchNorm 权重增加 `mask` 与 adversarial noise:

$$\gamma' = (m + \delta)\gamma$$

其中 m 是可学习 neuron mask, δ 是内层最大化阶段搜索到的神经元扰动。未加入 noise 时使用 $m\gamma$, 加入 noise 时使用 $(m + \delta)\gamma$ 。

3.2 Mask Optimization

generate_mask.py 中的 mask_train 使用 ANP 的 min-max 思路:

1. 对每个 batch 先随机初始化 neuron noise;
2. 内层优化固定 mask, 最大化带 noise 模型的分类损失, 得到较坏的 neuron perturbation;
3. 外层优化固定 noise, 同时最小化 clean loss 和 perturbed loss:

$$L = L_{clean} + \alpha_{anp} L_{perturbed}$$

4. 每次更新后把 mask clip 到 $[0, 1]$, 把 noise clip 到 $[-\epsilon, \epsilon]$ 。

这样得到的 mask value 越小, 说明对应神经元在 adversarial perturbation 下越可疑, 后续可以优先剪枝。

3.3 Threshold Pruning

prune_network.py 读取 mask_out/mask_values.txt, 按 mask score 从小到大排序。给定阈值 threshold 后, 所有 score 不超过阈值的 BatchNorm channel 会被置零, 然后分别在 clean test set 与 poison test set 上评估 ACC 和 ASR。

调参原则如下:

- anp_alpha 增大时, mask 优化会更关注 noisy / perturbed loss, 通常能更强地暴露后门相关神经元, ASR 更容易下降; 但过大时会误伤正常特征, 导致 clean ACC 下降。
- threshold 增大时, 剪掉的神经元更多, ASR 通常下降; 但阈值过大也会删除正常分类需要的通道, 使 clean ACC 下降。
- 实验时应先固定 anp_alpha=0.2 生成 mask, 再扫 threshold, 例如 0.02, 0.05, 0.08, 0.10。若 ASR 仍高于 5%, 再提高 anp_alpha 到 0.3 或 0.4 重新生成 mask。

4. 运行命令

在安装有 torch、torchvision 且能访问 CIFAR-10 的环境中, 可直接运行:

```
cd backdoor
./run_homework3.sh
```

也可以分步运行:

```
python generate_clb_attack.py
python train_backdoor.py --poison-type badnets --trigger-alpha 1.0
python train_backdoor.py --poison-type blend --trigger-alpha 0.2
python train_backdoor.py --poison-type clean-label --trigger-alpha 1.0
python generate_mask.py --anp-alpha 0.2
python prune_network.py --threshold 0.05
```

5. 本地验证记录

我在当前机器上完成了代码级验证:

- py_compile 通过: poison_cifar.py、generate_clb_attack.py、generate_mask.py、prune_network.py、train_backdoor.py、resnet_cifar.py、anp_batchnorm.py;
- 对 trigger 生成、训练投毒、测试投毒和 PGD helper 做了轻量单元验证;
- 对 mask_train 做了 toy NoisyBatchNorm 模型验证, 确认内层 noise 搜索与外层 mask 更新能正常执行。

当前本机 Python 环境缺少 torchvision, 且没有 CUDA, 因此没有在本机跑完整 CIFAR-10 训练和 ANP sweep; 提交包中保留了完整运行脚本, 日志目录用于保存正式训练输出。

6. 总结

本实现覆盖了 PPT 要求的两部分：后门攻击中补全三类 trigger、poison train/test 数据构造和 Clean-Label PGD；后门防御中补全 ANP mask 训练与 threshold pruning。攻击侧通过 PoisonACC 衡量 ASR，防御侧通过 sweep `anp_alpha` 与 `threshold` 在降低 ASR 和保持 clean ACC 之间折中。